



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Artificial and Computational Intelligence

AIMLCZG557

ACI Team



AIMLCZG557 – CS#7

Neural Architecture Search (NAS) and Game-Playing (Adversarial Search)

Agenda for CS #7

- 1) Recap of CS #6
- 2) Complete NAS (NEAT and CoDeepNEAT)
- 3) Introduction to Game playing
 - 1) Adversarial Settings?
 - 2) Game tree intuition
 - 3) Types of Games
 - 4) Static evaluation functions
 - 5) Introduction to Minimax algorithm
- 4) Q&A!



Phenotype Construction – Concept & Process

What is Phenotype Construction?

- Phenotype = *final* deep neural network
- Built from genotype (blueprint + modules)
- Used for training and fitness evaluation

Key Idea

- Small modules are assembled like building blocks
 - Creates a large, deep neural network automatically
-



Phenotype Construction – Concept & Process

1. Select a Blueprint

- Defines overall structure (flow of data)

2. Identify Nodes in Blueprint

- Each node = module placeholder (species ID)

3. Select Modules

- For each node, pick a module from its subpopulation

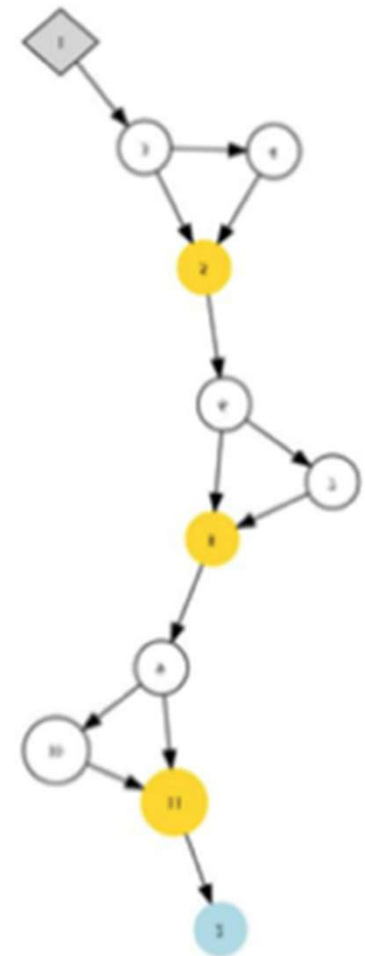
4. Replace Nodes with Modules

- Each node is expanded into a small neural subnetwork

5. Connect Modules

- Connections follow blueprint topology

Assembled Network



Phenotype Construction – Concept & Process

◆ Top Node (Input)

- Starting point of the network

● White Nodes

- Represent intermediate connections and flow

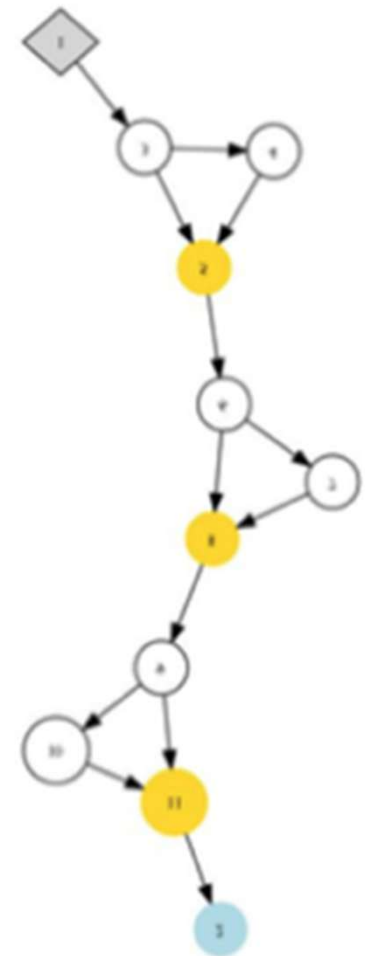
● Yellow Nodes (Important)

- These indicate module insertions
- Each yellow node = expanded module (subnetwork)

● Bottom Node (Output)

- Final prediction/output layer

Assembled Network



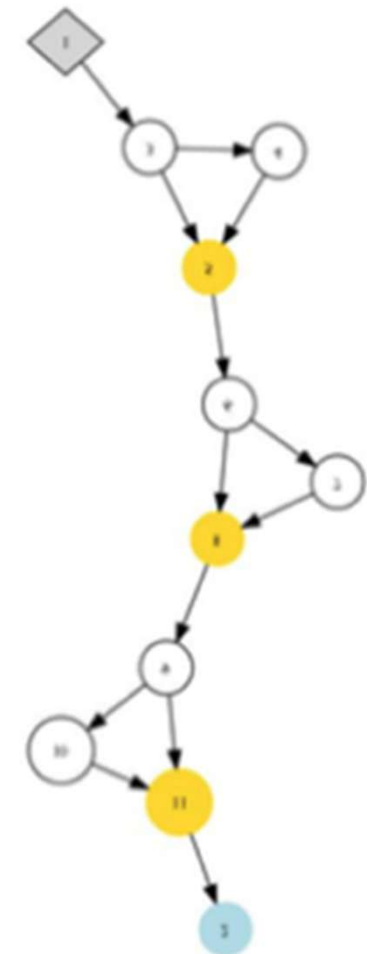


Phenotype Construction – Concept & Process

What is Happening Here?

- Each yellow node was originally a single blueprint node
- It is now replaced by a complex module
- These modules are:
 - ❖ Reused
 - ❖ Repeated at different positions
- Connections between them follow the blueprint structure

Assembled Network





Initialization – The Big Bang

- Evolution begins with a population of randomly generated, simple neural network architectures.
 - Example: Initialize 100 networks with minimal layers and basic connectivity.
 - These initial networks act as the starting point for evolutionary search.
 - Starting with simple structures allows gradual complexity growth through mutations and crossover.
 - This approach promotes efficiency and avoids unnecessary complexity in early stages.
-

The Evolutionary Mechanics

Fitness Evaluation Phase



- Each candidate architecture is instantiated as a neural network and trained using gradient descent.
 - Example: A CNN architecture is trained on a cat vs dog dataset containing 10,000 labeled images.
 - Training is performed for a limited number of epochs (e.g., 5–10 epochs) to evaluate learning capability.
 - The objective is to identify architectures that achieve higher accuracy within limited training time.
-



Defining Fitness (Performance Metric)

- Fitness is computed as classification accuracy on a validation dataset.
 - Example: If the network correctly classifies 90 out of 100 validation images, $\text{fitness} = 90\%$.
 - This ensures evaluation is based on generalization, not memorization.
 - Higher fitness values indicate better architectural suitability for the task.
-

Selection

Fitness-Proportionate Selection Mechanism



- Parent architectures are selected using a probabilistic approach based on fitness values.
- Example:
 - Network A (90% accuracy) → higher selection probability
 - Network B (50% accuracy) → lower selection probability
- Selection ensures that better-performing architectures are more likely to reproduce.



Detailed Selection Example

- Consider four architectures with fitness values:
 - A: 80%, B: 60%, C: 40%, D: 20%
 - Selection probabilities are proportional to these values.
 - Example outcome: A and B are selected multiple times; C occasionally; D rarely.
 - This maintains a balance between exploitation and exploration.
-



Crossover – Architectural Recombination

- Two selected parent architectures are combined to produce a new architecture.
 - Example:
 - Parent 1: Convolution → Pooling → Dense
 - Parent 2: Convolution → Convolution → Dense
 - Child: Convolution → Convolution → Dense (inherits deeper feature extraction capability)
-



Innovation Numbers in Crossover

- Each structural component is assigned an innovation number.
 - During crossover, matching components are aligned based on these identifiers.
 - Example:
 - Both parents contain a convolution layer with similar function → aligned
 - Extra layer in one parent → inherited if beneficial
 - This ensures valid and meaningful architecture generation.
-

Mutation

Structural and Parametric Variation



- Mutation introduces changes to explore new architectures.
- Types of mutation:
 - Add a new layer (e.g., additional convolution layer)
 - Add a connection (skip connection between layers)
 - Modify hyperparameters (e.g., filter size, activation function)
- Example: Changing filter size from 3×3 to 5×5 to capture broader features.



Detailed Mutation Example

- Original architecture: Convolution (3×3) $\rightarrow$ Pooling $\rightarrow$ Dense
 - Mutation: Convolution filter size changed to 5×5
 - Result: Improved ability to capture larger patterns such as full face structures
 - This may increase validation accuracy from 85% to 92%.
-



Speciation – Structural Grouping

- Architectures are grouped into species based on similarity metrics.
 - Example:
 - Species 1: shallow networks (1–2 layers)
 - Species 2: deeper networks (3–5 layers)
 - Competition occurs within species rather than globally.
 - This preserves diversity and allows new structures to evolve.
-



The Generational Evolution Process

- The evolutionary loop is repeated across multiple generations.
 - Each generation includes evaluation, selection, crossover, mutation, and speciation.
 - Example progression (cat vs dog classification):
 - Generation 1: ~50% accuracy (random performance)
 - Generation 20: ~75% accuracy (basic feature learning)
 - Generation 50: ~95% accuracy (advanced feature extraction)
 - Final architecture is selected based on highest validation performance.
-



Step 1: Input Image → Numerical Form

- Suppose we input a cat image (32×32 pixels)
 - Converted into a tensor:
 - $32 \times 32 \times 3 = 3072$ values
 - Example:
 - Pixel (R,G,B) = (120, 80, 60) → normalized → (0.47, 0.31, 0.23)
 - At this stage, the system does NOT know it is a cat - only numbers.
-



Step 2: Evolved Architecture is Selected

- CoDeepNEAT has already evolved a network like:
 - Module 1: Conv (3×3) + ReLU
 - Module 2: Conv (5×5) + Pool
 - Module 3: Dense layer
 - This structure was not manually designed; It was discovered through evolution.
-



Step 3: Forward Pass Begins (Feature Extraction)

Layer 1: Convolution (Edge Detection)

- Apply filter (3×3 kernel)
- Example output:
 - Detects edges of ears, body outline
- Output: Feature map highlighting edges

Layer 2: Activation (ReLU)

- Formula:

$$f(x) = \max(0, x)$$

- Removes negative values
- Keeps only important signals



Step 3: Forward Pass Begins (Feature Extraction)

Layer 3: Convolution (Texture Detection)

Combines edges → detects patterns

Example:

- Fur texture
- Eye regions

Now the network starts distinguishing cat-like texture

Layer 4: Pooling (Dimension Reduction)

Example: Max pooling (2×2)

Reduces size but keeps strongest features

Focus: “Is feature present?” not “Where exactly?”



Step 3: Forward Pass Begins (Feature Extraction)

Layer 5: Higher-Level Features

Combines patterns into objects:

- Triangular ears
- Small nose
- Whisker lines

These are cat-specific features



Step 4: Fully Connected Layer (Decision Preparation)

All extracted features are flattened into a vector

Example:

$[0.2, 0.8, 0.1, 0.9, \dots]$

This layer learns relationships like:

“If whiskers + triangular ears $\rightarrow$ likely cat”





Step 5: Output Layer (Softmax Classification)

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Suppose output:

- Cat = 0.92
- Dog = 0.08

Final Prediction:

Cat (92% confidence)



Step 6: During Training (Backpropagation)

If prediction is wrong:

Example:

- True label: Dog
- Predicted: Cat

Error is calculated:

$$\text{Loss} = \text{Predicted} - \text{Actual}$$

Weights are updated using gradient descent



Step 7: Where CoDeepNEAT Helps

Instead of manually designing:

- Number of layers
- Filter sizes
- Connections

CoDeepNEAT evolves:

- Best combination of modules
 - Best topology
 - Best hyperparameters
-



Step 8: Real Example of Evolution Impact

During evolution:

Network A:

- Only detects edges → Accuracy: 60%

Network B:

- Detects fur + ears → Accuracy: 85%

Network C (after mutation):

- Adds skip connection → Accuracy: 92%

Evolution selects Network C as the best



Step 9: Final Model Behavior

When deployed:

1. Input image → converted to numbers
2. Passed through evolved architecture
3. Features extracted automatically
4. Output → Cat or Dog





Step 10: Key Insight

The system does not “see” animals like humans

It detects:

➤ Edges → textures → parts → object

CoDeepNEAT ensures:

➤ The best architecture is discovered automatically



Modules & Blueprints – Scaling in CoDeepNEAT

- Evolving an entire deep neural network as a single structure is computationally expensive and difficult to optimize.
 - CoDeepNEAT addresses this by decomposing the architecture into two evolving populations:
 - Modules: reusable subnetwork components
 - Blueprints: high-level structural graphs
 - **Example:** Instead of evolving one full CNN for cat vs dog classification, smaller subnetworks (feature extractors) and their arrangement are evolved separately.
 - This decomposition enables efficient exploration of deeper and more complex architectures.
-



Modules – Reusable Subnetwork Components

- A module represents a small neural network (e.g., a sequence of convolution, pooling, and activation layers).
 - Each module has its own topology and hyperparameters that evolve over generations.
 - Example (Cat vs Dog):
 - Module A: Convolution (3×3) $\rightarrow$ ReLU $\rightarrow$ Max Pooling (detects edges and textures)
 - Module B: Convolution (5×5) $\rightarrow$ ReLU (captures larger patterns like face regions)
 - These modules act as feature extractors and can be reused multiple times within a network.
-



Blueprints – High-Level Network Structure

- A blueprint is a graph where each node points to a module species instead of a specific layer.
 - It defines how modules are arranged and connected to form a complete architecture.
 - Example (Cat vs Dog):
 - Node 1 → Module A (edge detector)
 - Node 2 → Module B (pattern detector)
 - Node 3 → Fully connected layer
 - The blueprint determines the overall flow of data and interaction between modules.
-

Assembly Process

Constructing the Neural Network



- Step 1: Select a blueprint from the blueprint population.
 - Step 2: For each node in the blueprint, randomly select a module from the corresponding module species.
 - Step 3: Replace blueprint nodes with selected modules and connect them as defined.
 - Example:
 - Blueprint structure: [Module A → Module B → Output Layer]
 - Selected modules are inserted to form a full CNN
 - The assembled network is then trained and evaluated on the cat vs dog dataset.
-

Shared Fitness – Coevolution Mechanism



- Fitness is assigned based on the performance of the assembled network.
 - The fitness score is distributed back to both:
 - The blueprint used to assemble the network
 - All modules that participated in that network
 - Example:
 - Assembled network achieves 92% accuracy on validation data
 - The blueprint and all contributing modules receive this fitness contribution
 - This enables simultaneous improvement of module quality and blueprint design across generations.
-



Example

A company wants to automatically generate captions for uploaded images using .The system evolves:

- CNN modules for image feature extraction
- LSTM modules for text generation
- Blueprints for arranging modules

Consider the following modules:

Module ID	Structure
M1	CNN Feature Extractor
M2	LSTM Layer
M3	Dense + Softmax

Blueprint: $B1=[M1 \rightarrow M2 \rightarrow M3]$

Validation scores:

Network	BLEU Score
N1	0.71
N2	0.88
N3	0.64

Demonstrate the first iteration of:

1. Phenotype Construction
2. Fitness Evaluation
3. Selection
4. Crossover
5. Mutation



Step 1: Initialization

Generate initial blueprint and module populations.

Blueprint Population

$B1 = [M1 \rightarrow M2 \rightarrow M3]$

$B2 = [M1 \rightarrow M1 \rightarrow M3]$

Module Population

Module ID	Structure
M1	CNN Feature Extractor
M2	LSTM Layer
M3	Dense + Softmax



Step 2: Genotype Representation

Blueprint

Defines the overall network structure.

$$B1 = [M1 \rightarrow M2 \rightarrow M3]$$

Modules

Reusable subnetworks.

M1 $\rightarrow$ CNN Feature Extractor

M2 $\rightarrow$ LSTM Layer

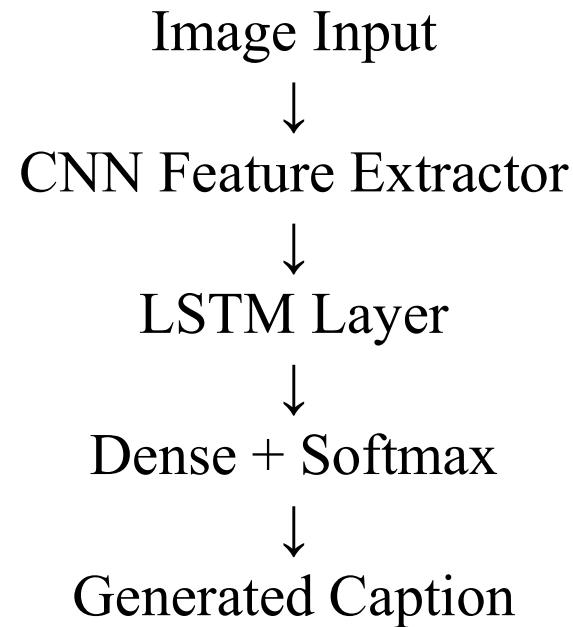
M3 $\rightarrow$ Dense + Softmax



Step 3: Phenotype Construction

Replace blueprint nodes with selected modules.

Constructed Architecture



This forms the final caption generation network.



Step 4: Fitness Evaluation

Fitness is computed using BLEU score.

Network	BLEU Score	Fitness
N1	0.71	0.71
N2	0.88	0.88
N3	0.64	0.64

Best Network:

$$N2 = 0.88$$

Hence N2 is selected as the strongest architecture.



Step 5: Selection

Using fitness proportionate selection:

(In fitness proportionate selection (roulette wheel selection):

- Higher fitness $\rightarrow$ higher probability of selection
- Lower fitness $\rightarrow$ lower probability)

Total Fitness:

$$0.71 + 0.88 + 0.64 = 2.23$$

$$\text{Probability of N1} = P(N1) = 0.71 / 2.23 = 0.318$$

$$\text{Probability of N2} = P(N2) = 0.88 / 2.23 = 0.394$$

$$\text{Probability of N3} = P(N3) = 0.64 / 2.23 = 0.287$$

Since:

- N2 has the highest probability
 - N1 has the second highest probability,
- they are selected as parent architectures for crossover.



Step 6: Crossover

Parent 1:

CNN $\rightarrow$ LSTM $\rightarrow$ Dense

Parent 2:

CNN $\rightarrow$ CNN $\rightarrow$ Dense

(Assume one-point crossover after the first module.)

Child 1

CNN $\rightarrow$ CNN $\rightarrow$ Dense

Child 2

CNN $\rightarrow$ LSTM $\rightarrow$ Dense



Step 7: Mutation

Mutation is applied to offspring architectures generated during crossover.

Before Mutation

Child 1: CNN $\rightarrow$ CNN $\rightarrow$ Dense

Child 2: CNN $\rightarrow$ LSTM $\rightarrow$ Dense

After Mutation

Child 1: CNN(3×3) $\rightarrow$ CNN(5×5) $\rightarrow$ Dense

Effect

- Captures larger image patterns
- Improves feature extraction capability

Child 2: CNN $\rightarrow$ LSTM(256) $\rightarrow$ Dense

Effect

- Improves sequence learning
- Better caption generation capability

Step 8: Add Mutated Offspring to Next Generation Population



The mutated children become part of the new population.

Example

Network	Architecture
Parent N1	CNN $\rightarrow$ LSTM $\rightarrow$ Dense
Parent N2	CNN $\rightarrow$ CNN $\rightarrow$ Dense
Mutated Child 1	CNN(3×3) $\rightarrow$ CNN(5×5) $\rightarrow$ Dense
Mutated Child 2	CNN $\rightarrow$ LSTM(256) $\rightarrow$ Dense

These architectures form the next generation population.



Step 9: Fitness Evaluation Again

All networks are trained and evaluated again.

Example

Network	Accuracy
Parent N1	88%
Parent N2	84%
Mutated Child 1	91%
Mutated Child 2	94%

Now

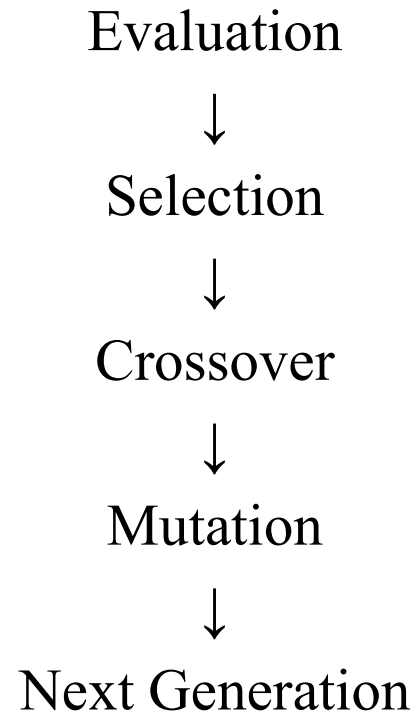
Mutated Child 2

becomes the best architecture.



Step 10: Evolution Continues

The process repeats:



Over many generations:

- weak architectures disappear
 - strong architectures survive
 - deeper and better networks evolve automatically
-

After several generations, CoDeepNEAT discovers



- optimal modules
- best blueprint structure
- efficient hyperparameters
- high-performing deep neural network architecture

This final network is deployed for tasks such as:

- image caption generation
- cat vs dog classification
- speech recognition
- medical image analysis.



Using CoDeepNEAT

- Modules and blueprints evolved separately
 - Better architectures were selected automatically using fitness evaluation
 - Two offspring architectures were generated through one-point crossover
 - Mutation modified module parameters and hyperparameters in offspring networks
 - Mutated offspring were added to the next generation population
 - Final evolved networks improved feature extraction and sequence learning capability
 - More accurate image captions were generated automatically
-

Exercise 1

A warehouse robot begins at C1, tasked with collecting Items (It) and reaching the Dock (DV) at C4 on a 4x4 grid. The robot must navigate hazards like blocked cells (#), Spills (SP) with high movement costs, while leveraging Energy packs (E) to reduce costs. Using the A* algorithm, the robot calculates the optimal path by considering both actual path cost and a heuristic. The agent has four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into blocked cells. The warehouse is represented as a 4×4 grid below:

	1	2	3	4
A	It	E		SP
B	SP	It	E	SP
C	S	E	It	DV
D	It	SP	SP	E

Entering SP: +12; collecting It: −8 (reward); energy pack E: +1.
Heuristic $h(n) = \text{Manhattan}(\text{robot}, \text{DV}) + (\# \text{ remaining Items}) + (\# \text{ adjacent SP to robot in resultant state})$.

- Expand and depict the search tree for exactly 2 levels. (Level 0 = Start).
- Calculate Path costs and heuristics (g/h/f) for all generated nodes.
- Apply A* search and show all the steps.

Exercise 2

1. Given two parent chromosomes:
 - Parent 1: [1, 2, 3, 4, 5, 6, 7, 8]
Parent 2: [8, 7, 6, 5, 4, 3, 2, 1]
 - Using single-point crossover after the 4th position, generate two children.
2. Suppose the fitness values of four individuals are:

Individual	Fitness
A	10
B	30
C	40
D	20

Which individual has the highest chance of being selected? Why?

Course Plan: Where are we in the Syllabus?



- M1 Introduction to AI
- M2 Problem Solving Agent using Search
- M3 Game Playing
- M4 Knowledge Representation using Logics
- M5 Probabilistic Representation and Reasoning
- M6 Reasoning over time
- M7 Ethics in AI



Learning Objectives of M3?

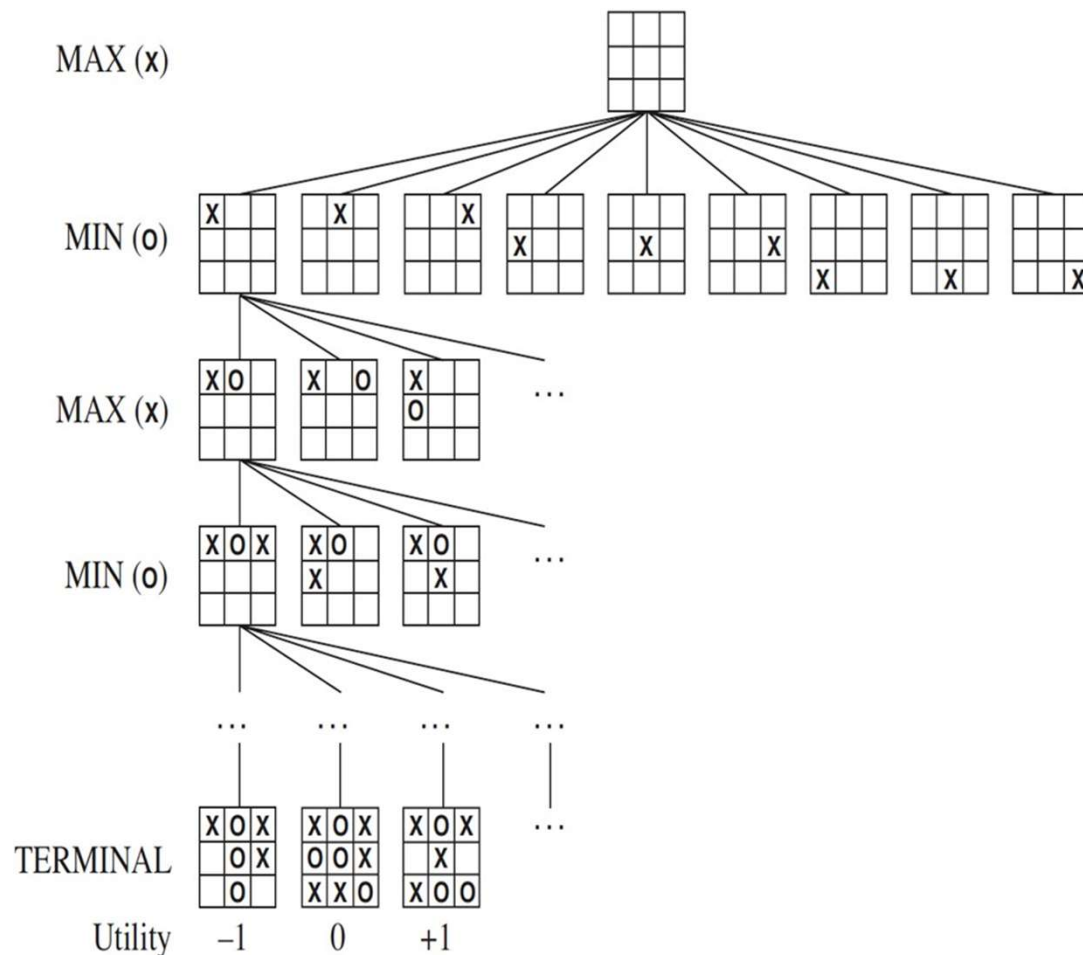
At the end of this class , you should be able to:

1. Convert a given problem into adversarial search problem
2. Formulate the problem solving agent components
3. Design static evaluation function value for a problem
4. Construct a Game tree
5. Apply Min-Max
6. Apply and list nodes pruned by alpha pruning and nodes pruned by beta pruning

Task Environment



Phases of Solution Search by PSA



Assumptions – Environment :

- Static**
- Observable**
- Discrete**
- Deterministic**
- Number of Agents**

Adversarial Search



- **Adversarial search** is search in a setting where two or more agents have conflicting goals.
- **Adversarial Search** is a branch of artificial intelligence (AI) focused on solving problems where two or more agents (players) have **opposing goals**. In such environments, one player's gain is another's loss.
- In such problems, an agent cannot plan only by asking: “*What action gets me closer to my goal?*”
- It must also ask: “*What will my opponent do to reduce my chance of success?*”
- *Examples: Chess, Tic-Tac-Toe, Go, Checkers, Competitive strategy games*

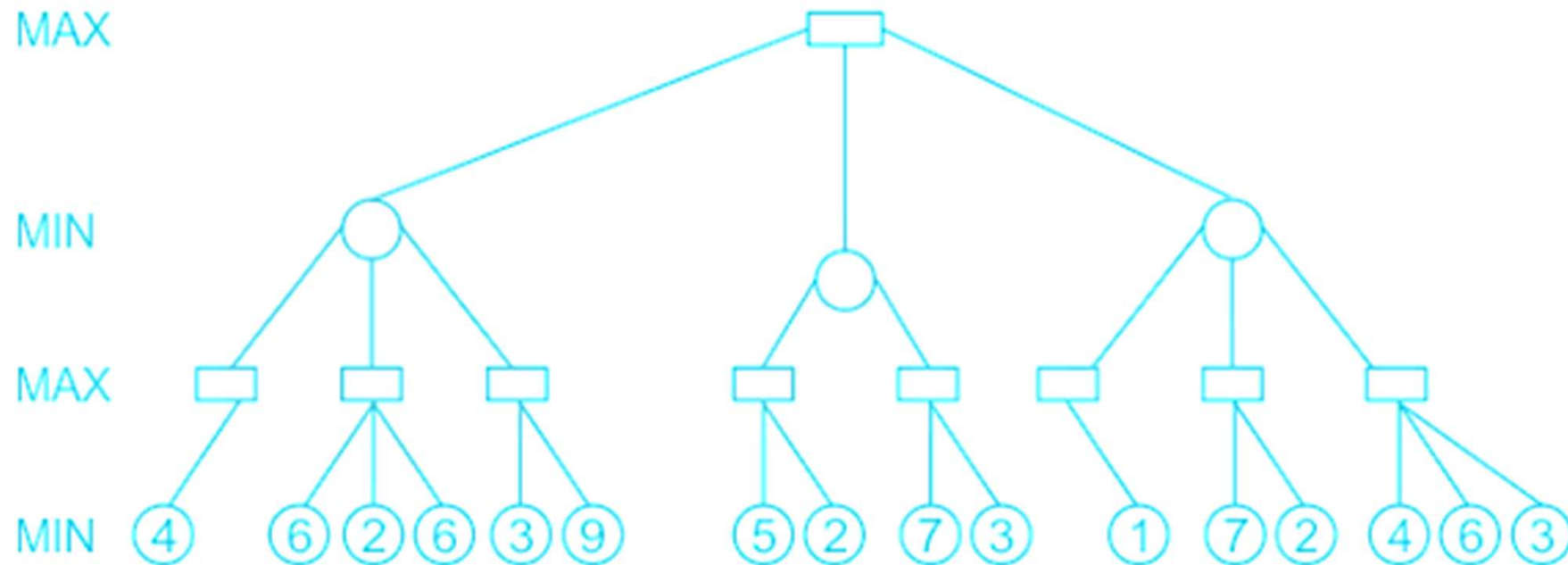




We will learn more about game playing in the next class ...

Exercise 1

Consider the following minimax game tree search



What will be the value propagated at the root?

ACI: Disclaimer and Acknowledgements

- Few content for these slides may have been obtained from prescribed books and various other source(s) on the Internet.
- We hereby acknowledge all the contributors for their material and inputs and gratefully acknowledge people others who made their course materials freely available online. We have provided source information wherever necessary.
- We have added and modified the content to suit the requirements of the class dynamics & live session's lecture delivery flow for presentation.
- This *is not a full-fledged* reading materials. Students are requested to refer to the textbook w.r.t detailed content as per the course handout as shared over e-learning portal - Taxila.
- **Slide Source / Preparation / Review:**
- From BITS Pilani WILP: Prof. Rajavadhana, Prof. Indumathi, Prof. Sangeetha and Prof Parthasarathy PD, Prof Ramya Devi
- From BITS On-campus & External : Mr.Santosh GSK



Thank You for your
time & attention !

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)

